

实验 5：黑盒与白盒测试在课程项目中的综合应用

项目	内容
课程项目名	WeaveDoc
小组名	TreeNewBee
2024302111237	谭博友
2024302111248	张启涵
2024302111363	任逸青
2024302111308	王志钢

目录

- 第零章：被测系统介绍
- 实验一：黑盒测试技术综合应用
 - 任务一：等价类划分法
 - 任务二：边界值分析法
 - 任务三：错误推测法
 - 任务四：判定表法
 - 任务五：因果图法
 - 任务六：正交试验法
- 实验二：白盒测试技术综合应用
 - 任务一：源代码结构分析
 - 任务二：绘制控制流图
 - 任务三：圈复杂度计算与基础路径识别
 - 任务四：六种覆盖标准测试用例设计
 - 任务五：JUnit 测试实现与缺陷报告
- 实验三：综合对比分析

第零章 被测系统介绍

0.1 项目简介

WeaveDoc 是一款 AI 辅助的结构化文档编辑器，主要面向需要撰写学术论文、技术报告 and 结构化文档的用户。核心功能包括：Markdown 实时编辑与预览、LaTeX 数学公式渲染、PDF 导入与导出、RAG 增强的 AI 写作助手，以及多标签页文档工作区管理。

0.2 被测模块说明

选取的类与方法：

类名	方法名（含参数列表）	返回值类型	核心职责（一句话）
MarkdownMathNormalizer	NormalizeDollarMath(string markdown)	string	对 Markdown 中的 $...$ 行内公式进行规范化处理，避免边界美元符号、代码块和转义符导致公式转换异常
MarkdownMathNormalizer	TryConvertTextCircledMath(string inner, out string circledText)	bool	识别 $\textcircled{n}$ 或 $\textcircled{\textcircled{n}}$ 形式的圈号公式，并转换为对应圈号字符
NativeMarkdownEditorControl	SetSelection(int start, int length)	void	设置编辑器选区，并对选区起点和长度进行边界修正，避免非法选区触发布局异常
NativeMarkdownEditorControl	GetSelection()	NativeMarkdownSelection	获取当前编辑器选区信息，兼容 AvaloniaEdit 编辑器和纯文本降级编辑器
NativeWebViewHost	NavigateToString(string html, Uri baseUri)	void	将 HTML 内容加载到跨平台 WebView 中，用于 Markdown 预览渲染
PreviewWebViewControl	ConvertHtmlToFallbackText(string html)	string	当 WebView 原生渲染不可用时，将 HTML 内容转换为纯文本预览内容

选取依据：

该模块满足本实验对被测对象的选取标准：

第一，该模块具有明确的输入验证和边界处理逻辑。

`MarkdownMathNormalizer.NormalizeDollarMath(string markdown)` 需要判断输入文本中的 `$` 是否构成合法行内公式边界，是否被反斜杠转义，是否属于 `$$` 块级公式，以及是否位于代码块中。`NativeMarkdownEditorControl.SetSelection(int start, int length)` 需要对选区起点和选区长度进行范围修正，防止负数、越界和超过文本长度的非法输入。

`PreviewWebViewControl.ConvertHtmlToFallbackText(string html)` 需要处理空 HTML、普通文本、HTML 标签、HTML 实体和多余空白等输入情况。

第二，该模块包含较多条件判断逻辑。公式预处理需要综合判断“是否在代码块内”“是否遇到单美元符号”“是否存在闭合美元符号”“公式内容是否为空”“是否为可转换的圈号公式”等条件。编辑器选区处理需要判断当前使用的是 AvaloniaEdit 编辑器还是纯文本降级编辑器、编辑器实例是否为空、选区范围是否合法。预览兼容处理需要判断原生 `WebView` 是否可用、当前平台渲染是否可靠、是否需要切换到纯文本回退视图。这些判断条件之间存在组合关系，适合使用判定表法、因果图法、条件覆盖和路径覆盖等方法进行测试设计。

第三，该模块具有明确可断言的输出。公式预处理方法返回规范化后的 `Markdown` 字符串，可以直接比较实际输出和期望输出。选区处理方法虽然没有直接返回值，但可以通过选区起点、选区长度、选中文本以及编辑器是否保持正常响应进行验证。`WebView` 预览兼容逻辑可以通过是否进入回退模式、回退文本内容是否正确、`WebView` 是否隐藏等状态进行断言。因此，该模块既适合黑盒测试中等价类划分、边界值分析和错误推测法，也适合白盒测试中判定覆盖、条件覆盖、条件组合覆盖和基础路径覆盖。

0.3 输入字段需求规格

输入字段	类型	有效范围或取值集合	备注
markdown	string	非 null 的 Markdown 文本；可为空字符串	用于公式预处理，文本中可能包含普通文本、行内公式 $\$...\$$ 、块级公式 $$$...$$$ 、转义美元符号 $\backslash \$$ 、代码块和行内代码
\$ 公式边界	字符	单个未转义 $\$$ ，且不是连续 $$$$	只有单美元符号才作为行内公式边界； $\backslash \$$ 和 $$$$ 不应作为普通行内公式处理
行内公式内容 inner	string	去除首尾空白后长度大于 0	空公式 $$$$ 不应被规范化为有效公式
圈号公式编号 n	int	$0 \leq n \leq 20$	$\text{\textcircled{n}}$ 或 $^{\text{\textcircled{n}}}$ 中的数字；超出范围不转换
代码块标记	string	<code>`</code> 或 <code>~</code>	位于代码块内部的内容不应进行公式规范化
start	int	$0 \leq \text{start} \leq \text{文本长度}$	编辑器选区起点；小于 0 时应修正为 0，大于文本长度时应修正到文本末尾
length	int	$0 \leq \text{length} \leq \text{文本长度} - \text{start}$	编辑器选区长度；小于 0 时应修正为 0，超过剩余文本长度时应截断
当前编辑器状态	枚举/布尔状态	AvaloniaEdit 编辑器、纯文本降级编辑器、编辑器未初始化	用于判断选区操作应作用于哪个编辑器对象
html	string	非 null 的 HTML 字符串；可为空字符串	用于 WebView 不可用时转换为纯文本回退内容
HTML 标签	string	合法或部分合法的 HTML 标签，如 <code><p></code> 、 <code>
</code> 、 <code></code>	转换回退文本时应去除标签，保留可读文本内容

HTML 实体	string	如 、<、>、&	回退文本中应解码为用户可读字符
WebView 可用状态	bool	true / false	当原生 WebView 不可用或渲染不可靠时，应进入纯文本回退模式

0.4 业务判断规则

条件定义：

条件编号	条件描述	满足（True）的定义
C1	当前文本位置处于代码块外	当前行不在 `` 或 ~~~ 包围的代码块内部
C2	当前 \$ 是合法行内公式边界	\$ 未被反斜杠转义，且不是 \$\$ 块级公式的一部分
C3	行内公式内容可被规范化	起止 \$ 之间内容去除首尾空白后不为空；若为 $\textcircled{n}$ 或 $^{\textcircled{n}}$ ，则 n 在 0~20 范围内

输出规则（优先级从高到低）：

优先级	条件组合	系统输出（期望行为）
1	C1 = F	保持原文不变，不处理代码块中的 \$...\$
2	C1 = T, C2 = F	保持原文不变，不把 \\$ 或 \$\$ 识别为普通行内公式
3	C1 = T, C2 = T, C3 = F	保持原文不变，避免空公式或非法圈号公式被错误转换
4	C1 = T, C2 = T, C3 = T, 且内容为普通公式	去除公式内容首尾空白，输出规范化的 \$公式内容\$

5	C1 = T, C2 = T, C3 = T, 且内容为合法圈号公式	将 $\textcircled{n}$ 或 $^{\textcircled{n}}$ 转换为对应圈号字符，如 ①、②、③
---	------------------------------------	--

补充业务规则：

对于编辑器选区处理，start 和 length 必须被限制在当前文本长度范围内；当输入为负数或超过文本长度时，系统应自动修正到合法范围，避免编辑器进入异常布局状态。

对于 WebView 预览处理，当原生 WebView 不可用或平台渲染不可靠时，系统应切换到纯文本回退预览，保证用户仍能看到文档内容，而不是显示空白或卡死。

0.5 被测方法源代码

```
namespace WeaveDoc.Converter.Pandoc;

internal static class MarkdownMathNormalizer
{
    private static readonly System.Text.RegularExpressions.Regex TextCircledOnlyRegex =
        new(@"^\textcircled{s*}\{s*(d{1,2})s*\}$", System.Text.RegularExpressions.RegexOptions.Compiled);

    private static readonly System.Text.RegularExpressions.Regex SuperscriptTextCircledOnlyRegex =
        new(@"^\^s*\{s*\textcircled{s*}\{s*(d{1,2})s*\}s*\}$", System.Text.RegularExpressions.RegexOptions.Compiled);

    public static string NormalizeDollarMath(string markdown)
    {
    }
```

```
var lines = markdown.Replace("\r\n", "\n").Split("\n");
```

```
var inFence = false;
```

```
string? fenceMarker = null;
```

```
for (var i = 0; i < lines.Length; i++)
```

```
{
```

```
    var trimmed = lines[i].TrimStart();
```

```
    if (StartsFence(trimmed, out var marker))
```

```
{
```

```
    if (!inFence)
```

```
{
```

```
        inFence = true;
```

```
        fenceMarker = marker;
```

```
}
```

```
    else if (marker == fenceMarker)
```

```
{
```

```
        inFence = false;
```

```
        fenceMarker = null;
```

```
}
```

```
    continue;
```

```
}
```

```
if (!inFence)
```

```

        lines[i] = NormalizeLine(lines[i]);

    }

    var normalized = string.Join("\n", lines);

    return markdown.Contains("\r\n", StringComparison.Ordinal) ? normalized.Replace("\n", "\r\n") : normalized;

}

private static string NormalizeLine(string line)

{

    var result = new System.Text.StringBuilder(line.Length);

    var i = 0;

    while (i < line.Length)

    {

        if (line[i] == "")

        {

            var end = FindInlineCodeEnd(line, i);

            if (end > i)

            {

                result.Append(line, i, end - i + 1);

                i = end + 1;

                continue;

            }

        }

    }

}

```

```

if (line[i] == '$' && !IsEscaped(line, i) && !IsDoubleDollar(line, i))

{

    var end = FindClosingDollar(line, i + 1);

    if (end > i)

    {

        var inner = line[(i + 1)..end];

        var trimmed = inner.Trim();

        if (trimmed.Length > 0)

        {

            if (TryConvertTextCircledMath(trimmed, out var circledText))

            {

                result.Append(circledText);

                i = end + 1;

                continue;

            }

            result.Append('$');

            result.Append(trimmed);

            result.Append('$');

            i = end + 1;

            continue;

        }

    }

}

```

```

        result.Append(line[i]);

        i++;
    }

    return result.ToString();
}

private static bool TryConvertTextCircledMath(string inner, out string circledText)
{
    circledText = "";

    var match = TextCircledOnlyRegex.Match(inner);

    if (!match.Success)

        match = SuperscriptTextCircledOnlyRegex.Match(inner);

    if (!match.Success || !int.TryParse(match.Groups[1].Value, out var number))

        return false;

    circledText = ToCircledNumber(number);

    return circledText.Length > 0;
}

private static string ToCircledNumber(int number)

```

```
{

    if (number == 0)

        return "u24EA";

    if (number is >= 1 and <= 20)

        return char.ConvertFromUtf32(0x245F + number);

    return "";

}
```

```
private static bool StartsFence(string trimmedLine, out string marker)
```

```
{

    marker = "";

    if (trimmedLine.StartsWith("```", StringComparison.Ordinal))

    {

        marker = "```";

        return true;

    }

}
```

```
if (trimmedLine.StartsWith("~~~", StringComparison.Ordinal))
```

```
{

    marker = "~~~";

    return true;

}
```

```
        return false;

    }

private static int FindInlineCodeEnd(string line, int start)

{

    var tickCount = 0;

    while (start + tickCount < line.Length && line[start + tickCount] == '"')

        tickCount++;

    for (var i = start + tickCount; i <= line.Length - tickCount; i++)

    {

        var matched = true;

        for (var j = 0; j < tickCount; j++)

        {

            if (line[i + j] != '"')

            {

                matched = false;

                break;

            }

        }

        if (matched)

            return i + tickCount - 1;

    }

}
```

```

    }

    return -1;

}

private static int FindClosingDollar(string line, int start)

{

    for (var i = start; i < line.Length; i++)

    {

        if (line[i] == '$' && !IsEscaped(line, i) && !IsDoubleDollar(line, i))

            return i;

    }

    return -1;

}

private static bool IsDoubleDollar(string line, int index) =>

    (index > 0 && line[index - 1] == '$') || (index + 1 < line.Length && line[index + 1] == '$');

private static bool IsEscaped(string line, int index)

{

    var slashCount = 0;

    for (var i = index - 1; i >= 0 && line[i] == '\\'; i--)

        slashCount++;

}

```

```
        return slashCount % 2 == 1;

    }

}
```

实验一：黑盒测试技术综合应用

本章全部基于第零章需求规格设计，不依赖源代码实现细节。

任务一：等价类划分法（20 分）

步骤 1：等价类划分表（10 分）

字段： markdown 中的公式边界（**有效范围：** 位于代码块外、未转义、非\$\$的单个 \$...\$ 行内公式）

等价类编号	字段	类型	描述	典型代表值
EC-V01	公式边界	有效	代码块外存在一对合法单美元符号 \$...\$	这是公式 $x + 1$
EC-I01	公式边界	无效	\$ 被反斜杠转义，不应作为公式边界	价格为 \\$100
EC-I02	公式边界	无效	使用 \$\$...\$\$ 块级公式，不应按行内公式处理	$x + 1$
EC-I03	公式边界	无效	\$...\$ 位于代码块内部，不应被转换	<code>```n\$ x + 1 \$n```</code>
EC-I04	公式边界	无效	只有起始 \$，没有闭合 \$	这是公式 $x + 1$

字段： 行内公式内容 inner（**有效范围：** 去除首尾空白后内容非空；普通公式可规范化，合法圈号公式 n 范围为 0~20）

等价类	字段	类型	描述	典型代表值
-----	----	----	----	-------

编号				
EC-V02	公式内容	有效	普通非空公式，去除首尾空白后保留为 \$公式内容\$	$x + y$
EC-V03	圈号公式	有效	$\textcircled{n}$ ，且 $0 \leq n \leq 20$	$\textcircled{1}$
EC-V04	上标圈号公式	有效	$\textcircled{n}^{\textcircled{m}}$ ，且 $0 \leq n \leq 20$	$\textcircled{3}^{\textcircled{3}}$
EC-I05	公式内容	无效	起止 \$ 之间内容为空或只有空白	\$ \$
EC-I06	圈号编号	无效	圈号编号大于 20，无法转换为支持字符	$\textcircled{21}$

步骤 2：测试用例设计（8 分）

用例编号	输入（列出所有参数）	覆盖等价类	预期结果
TC-EC-01	markdown = "这是公式 $x + y$ "	EC-V01, EC-V02	输出 "这是公式 $x + y$ "，公式首尾空白被去除
TC-EC-02	markdown = "序号 $\textcircled{1}$ "	EC-V01, EC-V03	输出 "序号 ①"
TC-EC-03	markdown = "引用 $\textcircled{3}^{\textcircled{3}}$ "	EC-V01, EC-V04	输出 "引用 ③"
TC-EC-04	markdown = "价格为 $\$100$ "	EC-I01	输出保持为 "价格为 $\$100$ "
TC-EC-05	markdown = " $\$x + y$ "	EC-I02	输出保持为 " $\$x + y$ "
TC-EC-06	markdown = "```\n $x + y$ \n```"	EC-I03	代码块内内容保持不变，不转换为 $x + y$
TC-EC-0	markdown = "这是公	EC-I04	没有闭合 \$，输出保持

7	式 $x + y$ "		原文
TC-EC-08	markdown = "\$ \textcircled{21} \$"	EC-I06	编号超出 0~20, 无法转换为圈号字符, 输出保持原公式或不转换为圈号字符

步骤 3: 分析 (2 分)

在本模块中, 最容易被遗漏的无效等价类是转义美元符号、代码块内公式和 $\$ \dots \$$ 块级公式。原因是这些输入表面上都包含 $\$$ 字符, 如果只按照“遇到 $\$$ 就寻找下一个 $\$$ ”的简单规则处理, 就可能把不应转换的内容错误识别为行内公式。

例如, $\$100$ 表示普通文本中的美元符号, 不是公式边界; 代码块中的 $x + y$ 是示例代码内容, 也不应被修改; $\$ x + y \$$ 属于块级公式, 不应按普通行内公式去除空白或转换。若忽略这些无效等价类, 可能导致 Markdown 原文语义被破坏, 进而影响预览和导出结果。

此外, 圈号公式的编号范围也容易遗漏。0~20 是可转换范围, 21 或非数字内容虽然匹配形式接近合法输入, 但不能生成有效圈号字符, 因此需要单独作为无效等价类测试。

任务二: 边界值分析法 (20 分)

步骤 1: 边界测试点 (12 分)

字段	有效范围	下界-1	下界	下界+1	上界-1	上界	上界+1
圈号公式编号 n	$0 \leq n \leq 20$	-1	0	1	19	20	21
去空白后公式内容长度	≥ 1	不适用	0	1	不设上界	不设上界	不设上界

行内公式闭合 \$ 数量	至少一对单美元符号	1	2	3	不设上界	不设上界	不设上界
-----------------	-----------	---	---	---	------	------	------

思考题：你的字段是否存在步长、枚举或格式约束？如有，如何定义该约束的“边界”？

本模块存在格式约束和数值边界约束。圈号公式必须符合 `\textcircled{n}` 或 `^{\textcircled{n}}` 的格式，其中 `n` 必须是数字，且有效范围为 0~20，因此 0 和 20 是有效边界，-1 和 21 是无效边界。公式内容需要在去除首尾空白后非空，因此长度 0 是无效边界，长度 1 是最小有效边界。行内公式必须由一对未转义的单美元符号包围，因此只有一个 \$ 是无效边界，两个 \$ 是最小合法情况。

步骤 2：边界值测试用例（6 分）

用例编号	字段及输入值	预期结果	说明
TC-BV-01	markdown="\$ \textcircled{-1} \$"	不转换为圈号字符，保持原公式内容	圈号编号下界-1
TC-BV-02	markdown="\$ \textcircled{0} \$"	输出 ①	圈号编号下界
TC-BV-03	markdown="\$ \textcircled{1} \$"	输出 ①	圈号编号下界+1
TC-BV-04	markdown="\$ \textcircled{20} \$"	输出 ②①	圈号编号上界
TC-BV-05	markdown="\$ \textcircled{21} \$"	不转换为圈号字符，保持原公式内容	圈号编号上界+1
TC-BV-06	markdown="\$ \$"	保持原文，不进行公式规范化	去空白后内容长度为 0
TC-BV-07	markdown="\$ x \$"	输出 \$x\$	最小非空公式内容
TC-BV-08	markdown="\$ x"	保持原文，不处	只有起始 \$，没有

		理	闭合边界
--	--	---	------

步骤 3：与等价类划分对比（2 分）

等价类划分主要关注输入属于哪一类，例如普通公式、圈号公式、转义美元符号、代码块内公式、空公式和非法圈号公式。它能保证测试覆盖主要输入类型，但不一定覆盖每个类型内部最容易出错的临界点。

边界值分析补充了这些临界情况。例如，圈号公式的有效编号范围是 0~20，等价类划分只会区分“范围内”和“范围外”，而边界值分析会进一步测试 0、1、20、21，可以发现 ≤ 20 被误写成 < 20 ，或 ≥ 0 被误写成 > 0 的问题。

公式内容长度也是类似。等价类划分会区分“空公式”和“非空公式”，边界值分析则重点测试长度 0 和长度 1，验证系统是否正确处理最小合法公式。对于 \$ 边界，只有一个 \$ 时不应处理，两个 \$ 才构成最小合法行内公式，这类临界输入也需要通过边界值测试补充验证。

任务三：错误推测法（10 分）

序号	推测的错误情形	测试输入（具体值）	预期输出	经验依据
EG-01	转义美元符号被误识别为公式边界	markdown="价格为 \\\$100，不是公式"	输出保持原文，不应改写为公式	\$ 在 Markdown 中既可表示公式，也可作为普通字符；若忽略反斜杠转义，容易误判
EG-02	块级公式被误当作行内公式处理	markdown="\$ \$ x + y \$ \$"	输出保持原文，不应去除 \$ \$ 或变成行内公式	\$ \$...\$ \$ 与 \$...\$ 只差一个字符，是公式解析中常见误判点
EG-03	代码块中的公式示例被错误	markdown="```\n\$ x + y \$ \n```"	代码块内 \$ x + y \$ 保持不变	Markdown 代码块应按原文展示，预处理逻辑如果未维护代码

	转换			块状态就会误改
EG-04	未闭合 \$ 导致后续正文被错误处理	markdown="公式开始 \$ x + y 后面是正文"	输出保持原文，不应删除、吞并或重排后续正文	未闭合符号是文本解析中常见异常输入，容易造成解析范围扩大
EG-05	圈号编号超出范围仍被转换	markdown="\$ \textcircled{21} \$"	不转换为圈号字符，保持原公式内容	系统只支持 0~20 的圈号字符，边界外编号容易导致空输出或错误字符
EG-06	上标圈号公式未被识别	markdown="引用 \$ ^{\textcircled{3}} \$"	输出 "引用 ③"	代码同时支持普通圈号和上标圈号两种格式，测试时容易只覆盖普通格式

分析：

错误推测法主要基于 Markdown 文本处理中的常见缺陷经验设计测试用例。本模块的核心风险在于 \$ 字符的语义判断，因为它既可能表示行内公式，也可能表示块级公式、普通字符或代码内容。因此，转义符号、块级公式、代码块和未闭合公式都是容易出错的输入场景。

此外，圈号公式转换属于格式识别逻辑，容易在编号边界和特殊写法上出现遗漏。例如，`\textcircled{21}` 虽然格式接近合法输入，但编号超出支持范围，不应被转换；`^{\textcircled{3}}` 是上标圈号格式，如果只测试普通 `\textcircled{3}`，就可能遗漏该场景。通过这些错误推测用例，可以补充等价类划分和边界值分析未完全覆盖的实际风险。

任务四：判定表法（20 分）

步骤 1：完整判定表（12 分）

本任务选取 MarkdownMathNormalizer.NormalizeDollarMath 中“是否处理 \$...\$ 行内公式”的核心规则建立判定表。根据实际逻辑选取 3 个主要条件，因此共有 $2^3 = 8$ 种规则。

条件说明：

条件编号	条件描述
C1	当前内容是否位于代码块外
C2	当前 \$ 是否为合法单美元公式边界，即未转义且不是 \$\$
C3	起止 \$ 之间的公式内容去空白后是否非空

动作说明：

动作编号	动作描述
A1	保持原文不变
A2	将普通行内公式规范化为 \$公式内容\$
A3	将合法圈号公式转换为圈号字符

条件/动作	R1	R2	R3	R4	R5	R6	R7	R8
C1：代码块外	Y	Y	Y	Y	N	N	N	N
C2：合法单美元边界	Y	Y	N	N	Y	Y	N	N
C3：公式内容非空	Y	N	Y	N	Y	N	Y	N
A1：保持原文不变		√	√	√	√	√	√	√

A2: 规范化普通公式	√							
A3: 转换合法圈号公式	√							
条件/动作	R1	R2	R3	R4	R5	R6	R7	R8
C1: 代码块外	Y	Y	Y	Y	N	N	N	N

说明：R1 中如果公式内容是普通公式，则执行 A2；如果公式内容是合法圈号公式，则执行 A3。二者都属于“合法行内公式内容”的处理结果。

步骤 2：化简判定表（4 分）

根据优先级规则，代码块内内容一律保持原文；非法美元边界一律保持原文；公式内容为空也保持原文。因此可将多个规则合并。

条件/动作	R1	R2	R3	R4
C1: 代码块外	Y	Y	Y	N
C2: 合法单美元边界	Y	Y	N	—
C3: 公式内容非空	Y	N	—	—
A1: 保持原文不变		√	√	√
A2: 规范化普通公式	√			
A3: 转换合法圈号公式	√			

化简后共 4 条规则，原 8 条，减少 50%。

合并依据：

当 C1 为 N 时，无论 C2、C3 是否满足，内容都位于代码块内，系统都应保持原文不变，因此可合并为 R4。当 C1 为 Y 但 C2 为 N 时，当前 \$ 不是合法行内公式边界，无论公式内容是否非空，都不应处理，因此可合并为 R3。当

C1 和 C2 均为 Y 但 C3 为 N 时，说明公式内容为空，也应保持原文，因此为 R2。只有 C1、C2、C3 均为 Y 时，系统才进行公式规范化或圈号转换。

步骤 3：具体测试用例（4 分）

规则	输入（所有参数及具体值）	预期结果
R1	markdown="这是公式 \$ x + y \$"	输出 "这是公式 \$x + y\$"
R1	markdown="序号 \$ \textcircled{2} \$"	输出 "序号 ②"
R2	markdown="空公式 \$ \$"	输出保持原文 "空公式 \$ \$"
R3	markdown="价格为 \\$100"	输出保持原文 "价格为 \\$100"
R4	markdown="```\n\$ x + y \$\n```"	代码块内内容保持不变

任务五：因果图法（15 分）

步骤 1：因果图描述（8 分）

原因节点	含义
c1	当前内容位于代码块外
c2	当前 \$ 是合法单美元公式边界，即未转义且不是 \$\$
c3	起止 \$ 之间的公式内容去除首尾空白后非空
c4	公式内容是合法圈号公式，且编号在 0~20 范围内

结果节点（输出）：

结果节点	含义
e1	保持原文不变
e2	将普通行内公式规范化为 \$公式内容\$
e3	将合法圈号公式转换为圈号字符

中间节点：

- m1：输入构成合法行内公式，定义为 $c1 \wedge c2 \wedge c3$

逻辑关系（用标准符号描述）：

$$\neg c1 \rightarrow e1$$

$$c1 \wedge \neg c2 \rightarrow e1$$

$$c1 \wedge c2 \wedge \neg c3 \rightarrow e1$$

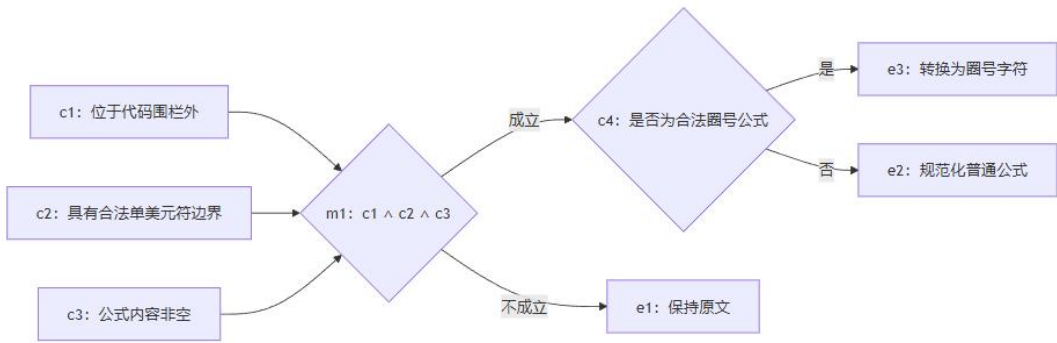
$$m1 \wedge c4 \rightarrow e3$$

$$m1 \wedge \neg c4 \rightarrow e2$$

约束关系说明：

e1、e2、e3 互斥，同一段输入只能产生一种主要处理结果。若内容位于代码块内、美元符号不是合法边界或公式内容为空，则优先保持原文不变。只有满足合法行内公式条件时，才进一步判断是否为合法圈号公式；若是合法圈号公式，则转换为圈号字符，否则按普通公式进行规范化。

控制流图或因果图示意（手绘后拍照嵌入，或用文字节点描述）：



2：导出测试场景（4 分）

场景	c1	c2	c3	c4
S1	T	T	T	T
S2	T	T	T	F
S3	T	T	F	—
S4	T	F	—	—

S5	F	—	—	—
场景	c1	c2	c3	c4

与任务四判定表核对：

因果图法导出的场景与任务四判定表一致。任务四判定表中，只有 C1=Y、C2=Y、C3=Y 时才会进入公式处理；因果图中对应中间节点 $m1 = c1 \wedge c2 \wedge c3$ 。在 m1 成立后，因果图进一步增加了 c4，用于区分普通公式规范化和圈号公式转换，因此比判定表更直观地表达了输出结果之间的互斥关系。

步骤 3：判定表法 vs 因果图法对比（3 分）

对比维度	判定表法	因果图法
适用场景	适合条件数量较少、规则组合明确的判断逻辑	适合条件之间存在因果关系、互斥关系或优先级关系的逻辑
条件约束表达能力	能列出条件组合和对应动作，但对条件之间的逻辑关系表达不够直观	能直观表达 与、或、非、互斥和中间条件
条件数量增加时的复杂度变化	条件增加时规则数按 2^n 增长，表格容易变大	可以通过中间节点合并逻辑，降低理解难度
核心优点	规则完整，便于检查是否遗漏条件组合	逻辑关系清晰，便于从原因推导结果
核心缺点	条件较多时表格冗长，阅读成本较高	需要先抽象原因和结果，绘制过程比判定表复杂

任务六：正交试验法（15 分）

选取的场景说明：

本任务选取 `MarkdownMathNormalizer.NormalizeDollarMath` 的 Markdown 公式预处理场景进行正交试验。该方法的输出同时受公式所在位置、美元符号类型、公式内容类型和文本换行形式影响。如果采用全组合测试，测试数量较多，因此使用 $L_9(3^4)$ 正交表选取代表性组合。

步骤 1：确定因素与水平，全组合基线（1 分）

因素	水平 1	水平 2	水平 3
因素 A：公式所在位置	普通正文	行内代码	代码块
因素 B：美元符号类型	合法单美元 \$...\$	转义美元 <code>\\$</code>	块级公式 <code>\$\$...\$\$</code>
因素 C：公式内容类型	普通公式 $x + y$	合法圈号 <code>\textcircled{1}</code>	非法圈号 <code>\textcircled{21}</code>
因素 D：文本换行形式	单行文本	Unix 换行 <code>\n</code>	Windows 换行 <code>\r\n</code>

全组合共需测试用例：81 个

步骤 2：应用 $L_9(3^4)$ 正交表（10 分）

用例编号	因素 A：公式所在位置	因素 B：美元符号类型	因素 C：公式内容类型	因素 D：文本换行形式	预期结果
OA-01	普通正文	合法单美元	普通公式	单行文本	<code>\$ x + y \$</code> 被规范化为 <code>\$x + y\$</code>
OA-02	普通正文	转义美元	合法圈号	Unix 换行	<code>\\$</code> 不作为公式边界，内容保持原文
OA-03	普通正文	块级公式	非法圈号	Windows 换行	<code>\$\$...\$\$</code> 不按行内公式处理，换行风格保持
OA-04	行内代码	合法单美元	合法圈号	Windows 换行	行内代码中的 <code>\$ \textcircled{1}</code> <code>\$</code> 保持原文

OA-05	行内代码	转义美元	非法圈号	单行文本	行内代码和转义美元均不触发公式转换
OA-06	行内代码	块级公式	普通公式	Unix 换行	行内代码中的块级公式内容保持原文
OA-07	代码块	合法单美元	非法圈号	Unix 换行	代码块内 \$ \textcircled{21} \$ 保持原文
OA-08	代码块	转义美元	普通公式	Windows 换行	代码块内内容保持原文, Windows 换行不被破坏
OA-09	代码块	块级公式	合法圈号	单行文本	代码块内 \$\$ \textcircled{1} \$\$ 保持原文

步骤 3：均匀性验证（2 分）

任选两个因素（如 $A \times B$ ），统计 9 个用例中每对组合出现次数：

A \ B	合法单美元	转义美元	块级公式
普通正文	1	1	1
行内代码	1	1	1
代码块	1	1	1

验证结论：在 9 个测试用例中，因素 A 的每个水平与因素 B 的每个水平都恰好组合 1 次，说明该正交表在这两个因素上具有均匀分布特征。其他任意两个因素之间也满足类似的均匀组合关系，因此能够用较少用例覆盖主要因素组合。

步骤 4：效率分析（2 分）

- 节省用例数量：72 个，减少比例：72 / 81 $\approx$ 88.9%%
- 正交试验能保证：
- 正交试验不能保证：

正交试验能保证：

在较少测试用例下，使不同因素及其水平组合分布较均匀，适合发现由多个输入因素共同影响导致的典型缺陷。例如，可以同时覆盖正文、行内代码、代码块，合法美元、转义美元、块级公式，以及普通公式、合法圈号、非法圈号等组合。

正交试验不能保证：

正交试验不能覆盖全部 81 种输入组合，也不能保证发现所有特定组合下才出现的缺陷。例如，如果某个缺陷只在“Windows 换行 + 普通正文 + 合法单美元 + 非法圈号”的特定组合下出现，而该组合没有被正交表选中，则可能无法被本组用例发现。因此，正交试验适合作为组合测试的高效抽样方法，但仍需要结合边界值分析和错误推测法补充关键风险场景。

实验二：白盒测试技术综合应用

任务一：源代码结构分析（5 分）

1. 公共方法与内部调用关系

公共方法	内部直接调用	间接调用
NormalizeDollarMath(string StartsFence(...), markdown)	NormalizeLine(...)	NormalizeLine 内部继续调用 FindInlineCodeEnd(...), IsEscaped(...), IsDoubleDollar(...), FindClosingDollar(...), TryConvertTextCircledMath(...); TryConvertTextCircledMath 调用 ToCircledNumber(...)
调用链		

```
NormalizeDollarMath
├─ StartsFence
├─ NormalizeLine
│   ├── FindInlineCodeEnd
│   ├── IsEscaped
│   ├── IsDoubleDollar
│   ├── FindClosingDollar
│   │   ├── IsEscaped
│   │   └─ IsDoubleDollar
├─ TryConvertTextCircledMath
└─ ToCircledNumber
```

2. 主方法中的判断语句

(列出所有 if 判断，每个判断的条件表达式是什么)

判断编号	条件表达式	含义描述
D1	StartsFence(trimmed, out var marker)	当前行是否是 Markdown 代码块围栏行，例如 ``` 或 ~~~
D2	!inFence	当前不在代码块内部，遇到围栏时进入代码块
D3	marker == fenceMarker	当前在代码块内，且遇到与开始围栏相同的结束围栏，退出代码块
D4	!inFence	当前行不在代码块内部，才对这一行执行数学公式规范化

tips:

主方法最后还有一个条件表达式，但它不是 if 语句：

```
markdown.Contains("\r\n", StringComparison.Ordinal)
```

它用于决定返回结果是否恢复为 Windows 风格换行符 \r\n。

3. 各辅助方法的职责

辅助方法	职责
StartsFence(string trimmedLine, out string	判断一行是否以代码块围栏开头，并

marker)	识别围栏类型是 `` 还是 ~~~
NormalizeLine(string line)	规范化单行内容中的单美元数学公式，跳过行内代码
FindInlineCodeEnd(string line, int start)	从反引号开始位置查找对应的行内代码结束位置
FindClosingDollar(string line, int start)	查找单美元数学公式的结束 \$
IsDoubleDollar(string line, int index)	判断当前位置的 \$ 是否属于 \$\$ 双美元公式
IsEscaped(string line, int index)	判断当前位置字符是否被反斜杠转义
TryConvertTextCircledMath(string inner, out string circledText)	尝试把 \textcircled{n} 或 ^{\textcircled{n}} 转成 Unicode 圆圈数字
ToCircledNumber(int number)	把 0 到 20 转成对应的 Unicode 圆圈数字，超出范围返回空字符串

4. 主方法可能返回的结果与对应路径

返回值	触发路径描述
normalized	输入文本不包含 \r\n。方法按 \n 拆分、处理非代码块行后，用 \n 拼接返回
normalized.Replace("\n", "\r\n")	输入文本包含 \r\n。方法内部统一按 \n 处理，最后恢复成 Windows 风格换行
内容可能与原文相同的字符串	没有可规范化的单美元公式，或相关内容都在代码块中，因此处理后文本语义上保持不变
内容被规范化后的字符串	非代码块行中存在 \$... \$ 单美元公式，内部空白会被 Trim()；若公式是 \textcircled{n} 形式，还会转换为圆圈数字

任务二：绘制控制流图（15 分）

2.1 主方法控制流图（8 分）

节点列表：

节点	内容
1	入口
2	初始化：统一换行、拆分行、inFence = false、fenceMarker = null
3	判断循环：i < lines.Length
4	处理：trimmed = lines[i].TrimStart()
5	判断 D1：StartsFence(trimmed, out var marker)
6	判断 D2：!inFence
7	处理：进入代码块，设置 inFence = true、fenceMarker = marker
8	判断 D3：marker == fenceMarker
9	处理：退出代码块，设置 inFence = false、fenceMarker = null
10	处理：continue，进入下一轮循环
11	判断 D4：!inFence
12	处理：lines[i] = NormalizeLine(lines[i])
13	处理：循环递增，进入下一轮
14	处理：normalized = string.Join('\n', lines)
15	判断 D5：markdown.Contains("\r\n", StringComparison.Ordinal)
16	返回：normalized.Replace("\n", "\r\n")
17	返回：normalized
18	出口

控制流边列表:

1→2

2→3

3→4 (T)

3→14 (F)

4→5

5→6 (T)

5→11 (F)

6→7 (T)

6→8 (F)

7→10

8→9 (T)

8→10 (F)

9→10

10→3

11→12 (T)

11→13 (F)

12→13

13→3

14→15

15→16 (T)

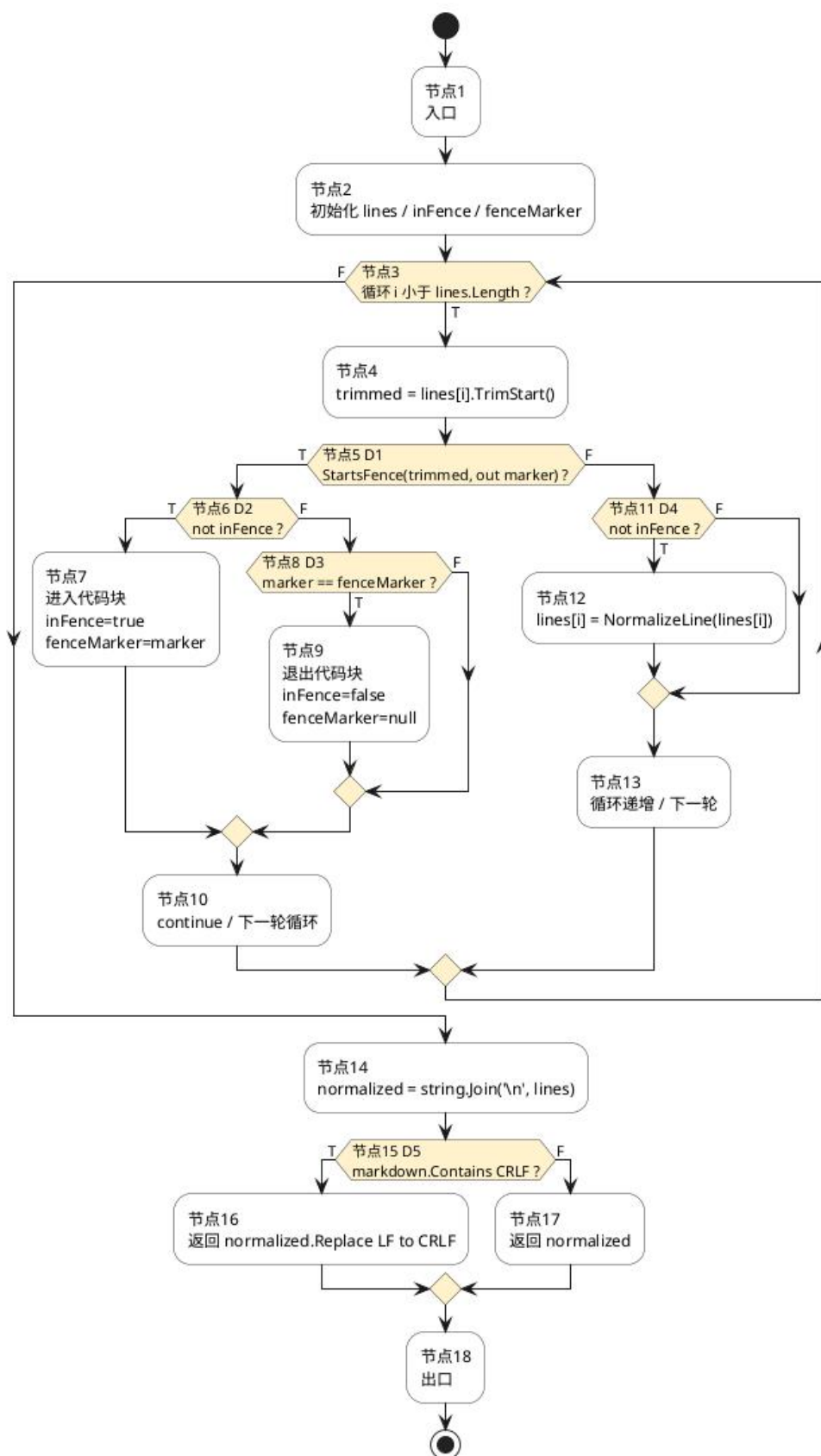
15→17 (F)

16→18

17→18

节点数 = 18, 边数 = 23

控制流图示意（手绘后拍照嵌入，或用文字节点描述）：



2.2 辅助方法控制流图（4 分）

选取方法：_选取方法： StartsFence(string trimmedLine, out string marker)

节点	内容
1	入口
2	处理： marker = ""
3	判断 D1: trimmedLine.StartsWith("```", StringComparison.Ordinal)
4	处理： marker = "```"
5	返回： true
6	判断 D2: trimmedLine.StartsWith("~~~", StringComparison.Ordinal)
7	处理： marker = "~~~"
8	返回： true
9	返回： false
10	出口

1→2

2→3

3→4 (T)

3→6 (F)

4→5

5→10

6→7 (T)

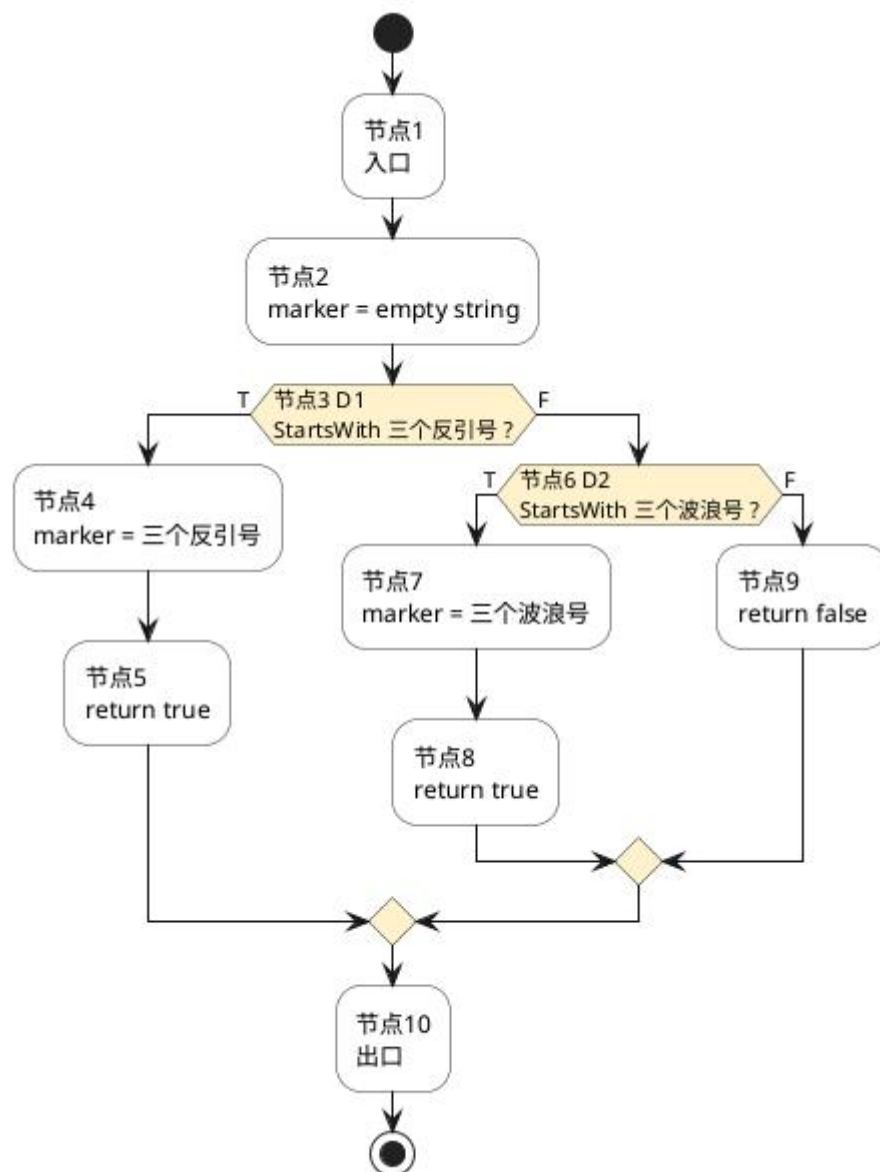
6→9 (F)

7→8

8→10

9→10

节点数 = 10，边数 = 11



2.3 含复合条件方法的控制流图（3 分）

选取方法：TryConvertTextCircledMath(string inner, out string circledText)

复合语句原句：

```
if (!match.Success || !int.TryParse(match.Groups[1].Value, out var number))
```

拆分：

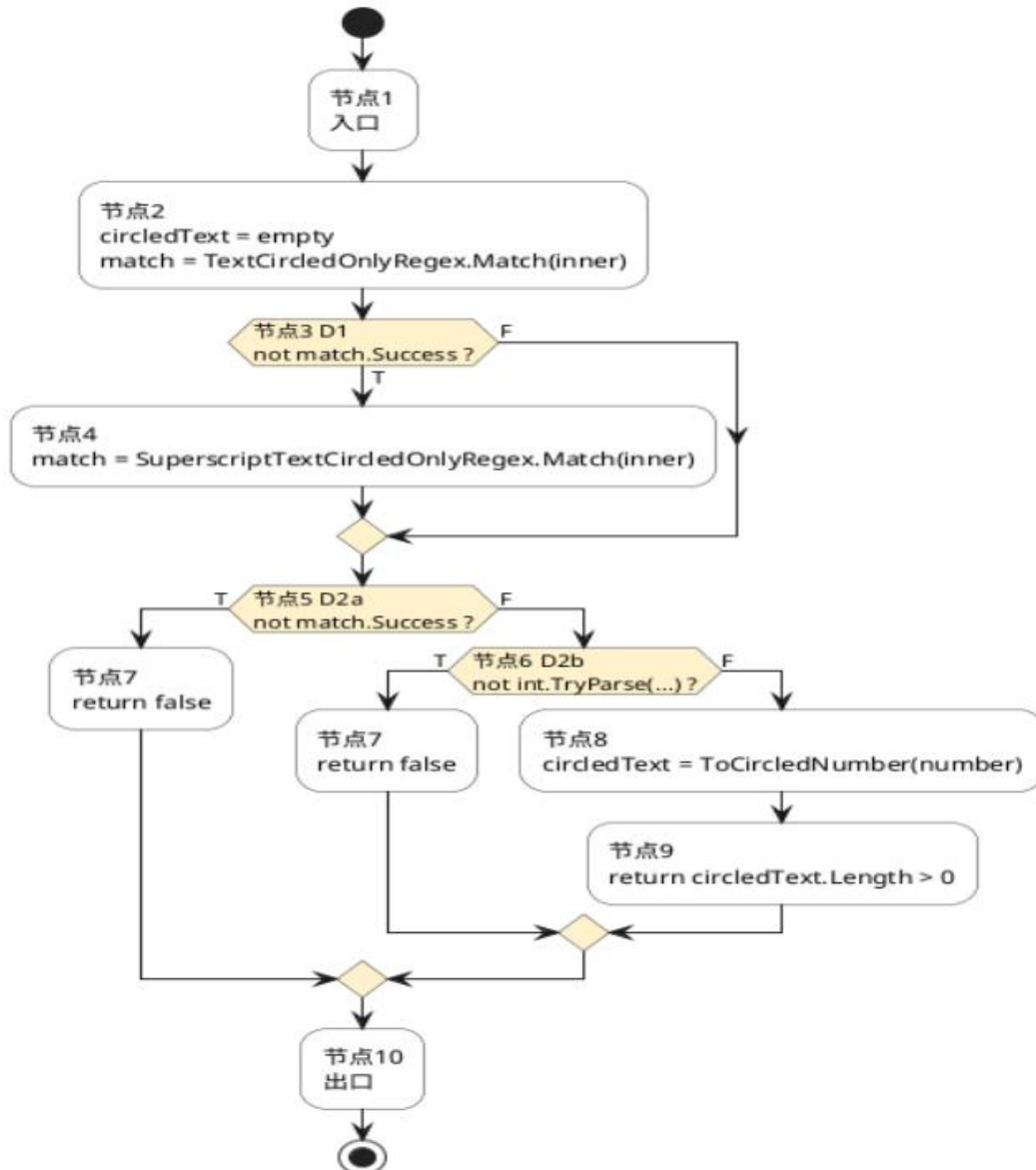
判断

条件表达式

D2a `!match.Success`

D2b `!int.TryParse(match.Groups[1].Value, out var number)`

节点数 = 10，边数 = 12



任务三：圈复杂度计算与基础路径识别（20 分）

3.1 圈复杂度计算（8 分）

方法名	判断节点数（含复合条件拆分）	V(G)	含义（最少独立测试路径数）
NormalizeDollarMath 主方法本体，按 D1-D5 口径	5	6	至少 6 条基础路径
StartsFence	2	3	至少 3 条基础路径
NormalizeLine	9	10	至少 10 条基础路径
TryConvertTextCircledMath	3	4	至少 4 条基础路径
ToCircledNumber	3	4	至少 4 条基础路径
FindInlineCodeEnd	6	7	至少 7 条基础路径
FindClosingDollar	4	5	至少 5 条基础路径
IsDoubleDollar	4	5	至少 5 条基础路径
IsEscaped	2	3	至少 3 条基础路径

展开辅助方法后，主方法综合 CFG 共 __ 个判断节点，V(G) = __，即共 __ 条基础路径。

3.2 基础路径识别（12 分）

编号	条件
D1	StartsFence(trimmed, out var marker)
D2	!inFence
D3	marker == fenceMarker
D4	!inFence
D5	markdown.Contains("\r\n", StringComparison.Ordinal)

路径 编号	D1	D2	D3	D4	D5	预期返回值（按需求规格）
P1	F	-	-	T	F	返回 <code>normalized</code> ，普通非代码块行被 <code>NormalizeLine</code> 处理，保留 <code>\n</code>
P2	F	-	-	T	T	返回 <code>normalized.Replace("\n", "\r\n")</code> ，普通非代码块行被处理，并恢复 CRLF
P3	T	T	-	-	F	返回 <code>normalized</code> ，遇到代码块开始围栏，进入代码块
P4	F	-	-	F	F	返回 <code>normalized</code> ，当前位于代码块内部，普通行不做数学公式规范化
P5	T	F	T	-	F	返回 <code>normalized</code> ，当前位于代码块内部，遇到匹配围栏，退出代码块
P6	T	F	F	-	F	返回 <code>normalized</code> ，当前位于代码块内部，遇到不同类型围栏，保持代码块状态

思考题：是否存在理论可列但实际不可达的路径？

存在理论可列但实际不可达的路径。

主要不可达情况：

理论组合	是否可达	原因
D1 = F 且继续判断 D2 或 D3	不可达	D2、D3 只在 D1 = T 的分支中出现
D1 = T 且继续判断 D4	不可达	D4 只在 D1 = F 的分支中出现
D2 = T 且继续判断 D3	不可达	D3 只在 D2 = F，即已经处于代码块内部时判断
首次循环前直接跳到最终返	实际不可	<code>markdown.Replace(...).Split('\n')</code> 至少产生一个元素，因此

回	达	lines.Length 至少为 1
单行输入中出现 D4 = F	不可达	D4 = F 表示已经在代码块内部，需要之前有未闭合的围栏行；单行普通输入无法满足
D3 = T 但之前没有进入代码块	不可达	D3 依赖 fenceMarker 已经由之前的开始围栏设置

任务四：六种覆盖标准测试用例设计（40 分）

重要：所有“预期结果”均依据第零章需求规格填写，而非依照代码实际输出。

4.1 语句覆盖（5 分）

可执行语句清单：

- S1: markdown.Replace("\r\n", "\n").Split('\n')
- S2: if (StartsFence(trimmed, out var marker))
- S3: if (!inFence)
- S4: else if (marker == fenceMarker)
- S5: if (!inFence) lines[i] = NormalizeLine(lines[i])
- S6: return markdown.Contains("\r\n", StringComparison.Ordinal) ?
normalized.Replace("\n", "\r\n") : normalized

用例编号	输入（所有参数）	预期结果（按规格）	覆盖的语句
SC-01	"正常文本"	"正常文本"	S1, S2(F), S5(T), S6
SC-02	"csharp\r\n 代码\r\n"	"csharp\r\n 代码\r\n"	S1, S2(T), S3(T), S4(F), S6
SC-03	"csharp\r\n 代码\r\n"	"csharp\r\n 代码\r\n"	S1, S2(T), S3(T), S4(T), S5(F), S6
SC-04	"文本\r\n 换行"	"文本\r\n 换行"	S1, S6 (T 分支)

不可达语句（若有）：

无。所有核心分支均可通过构造合适的输入到达。

分析：仅凭语句覆盖，哪类缺陷最可能被遗漏？

语句覆盖是最弱的逻辑覆盖。它仅保证每条代码被执行一次，但无法发现逻辑运算符错误（如 `&&` 错写成 `||`）、条件判断缺失或隐藏的逻辑分支缺陷。例如，如果 `StartsFence` 内部判断逻辑出错，只要有一条路径走通，语句覆盖依然显示 100%。

4.2 判定覆盖（5 分）

判断语句及 `True/False` 条件：

判断	True 条件示例	False 条件示例
D1: <code>StartsFence(...)</code>	输入包含 <code><</code> 或 <code>></code>	输入为普通文本
D2: <code>!inFence</code>	初始状态或代码块外	处于未闭合的代码块内
D3: <code>marker == fenceMarker</code>	闭合标记与开启标记相同	闭合标记与开启标记不同

用例编号	输入（所有参数）	预期结果	D		
			1	D2	D3
DC-01	"普通文本"	"普通文本"	F	-	-
DC-02	"csharp\r\n 代码"	"csharp\r\n 代码"	T	T	-
DC-03	"csharp\r\n 代码\r\n"	"csharp\r\n 代码\r\n"	T	F	T
DC-04	"csharp\r\n 代码\r\n~~~"	"csharp\r\n 代码\r\n~~~"	T	F	F

关键问题： 仅用（全 T）和（全 F）的输入组合，能否检出逻辑运算符错误（如 `t&&` 写成 `||`）？说明理由：

不能。因为全 T 或全 F 组合只覆盖了表达式的最终结果，没有覆盖到原子条件的中间状态。如果代码中 `A && B` 被误写为 `A || B`，在特定的全 T 或全 F 输入下，最终判定结果可能碰巧相同，从而掩盖了逻辑运算符的错误。

4.3 条件覆盖（8 分）

原子条件分析：

原子条件	含义	取 true 的测试输入	取 false 的测试输入
C1	StartsFence 返回 true	""	"abc"
C2	!inFence 为 true	初始状态	进入 Fence 后
C3	marker == fenceMarker	"" 对应 ""	"" 对应 "~~~"

用例编号	输入（所有参数）	各原子条件值	预期结果（规格）	实际结果	是否一致
CC-01	"普通文本"	C1=F	"普通文本"	"普通文本"	是
CC-02	"csharp\r\n 代码"	C1=T, C2=T	"csharp\r\n 代码"	"csharp\r\n 代码"	是
CC-03	"csharp\r\n 代码\r\n"	C1=T, C2=F, C3=T	"csharp\r\n 代码\r\n"	"csharp\r\n 代码\r\n"	是
CC-04	"csharp\r\n 代码\r\n~~~"	C1=T, C2=F, C3=F	"csharp\r\n 代码\r\n~~~"	"csharp\r\n 代码\r\n~~~"	是

分析问题 1： 实际结果与预期不符时，对应哪个代码缺陷？说明根本原因：

如果 CC-03 或 CC-04 不符，说明 StartsFence 方法未能正确识别闭合标记，或者 fenceMarker 的状态管理存在 Bug，导致代码块未能正确闭合，进而可能错误地对代码块内的数学公式进行了 Normalize 处理。

分析问题 2： 条件覆盖是否蕴含判定覆盖？用你的测试用例说明：

不蕴含。虽然 CC-01 到 CC-04 覆盖了所有原子条件的 T/F 取值，但如果不刻意构造，可能遗漏某些判定的整体分支。例如，如果只设计了满足 C1=T, C2=T

的用例，可能遗漏了 C1=T, C2=F 时 D3 的 False 分支。条件覆盖只保证原子条件被覆盖，不保证判定结果的所有分支都被覆盖。

4.4 判定-条件覆盖（5 分）

用例编号	输入（所有参数）	各原子条件值	判断整体值（T/F）	预期结果
DCC-01	"普通文本"	C1=F	D1=F	"普通文本"
DCC-02	"csharp\r\n 代码"	C1=T, C2=T	D1=T, D2=T	"csharp\r\n 代码"
DCC-03	"csharp\r\n 代码\r\n"	C1=T, C2=F, C3=T	D1=T, D2=F, D3=T	"csharp\r\n 代码\r\n"
DCC-04	"csharp\r\n 代码\r\n~~~"	C1=T, C2=F, C3=F	D1=T, D2=F, D3=F	"csharp\r\n 代码\r\n~~~"

补充说明：混合标记的临界值（如 DCC-04 中的 开启但用 ~~~ 闭合）。测试人员容易默认代码块一定是成对相同标记闭合，从而遗漏了不同标记混合时的状态机判断逻辑。

（填写分析）

4.5 条件组合覆盖（7 分）

组合编号	原子条件组合（T/F）值	判断期望值	具体输入	预期输出（规格）	实际输出
MCC-01	C1=T, C2=T	D1=T, D2=T	""`csharp\r\n 代码"	""`csharp\r\n 代码"	""`csharp\r\n 代码"
MCC-02	C1=T, C2=F, C3=T	D1=T, D2=F, D3=T	""`csharp\r\n 代码\r\n""	""`csharp\r\n 代码\r\n""	""`csharp\r\n 代码\r\n""
MCC-03	C1=T, C2=F, C3=F	D1=T, D2=F, D3=F	""`csharp\r\n 代码\r\n~~~"	""`csharp\r\n 代码\r\n~~~"	""`csharp\r\n 代码\r\n~~~"
MCC-04	C1=F	D1=F	"普通文本"	"普通文本"	"普通文本"

分析：在当前提供的代码逻辑下，实际结果与预期均一致。若出现不符，对应缺陷为 MarkdownMathNormalizer 中的状态机（inFence 和 fenceMarker）流转逻辑错误，导致未正确跳过代码块内的内容。

（填写分析）

4.6 基础路径覆盖（10 分）

路径	输入（所有参数）	路径经过的判断节点	预期返回值（规格）	实际返回值	是否发现缺陷
P1	"普通文本"	D1(F)	"普通文本"	"普通文本"	否
P2	"``csharp\r\n 代码"	D1(T), D2(T)	"``csharp\r\n 代码"	"``csharp\r\n 代码"	否
P3	"``csharp\r\n 代码\r\n``"	D1(T), D2(T), D1(T), D2(F), D3(T)	"``csharp\r\n 代码\r\n``"	"``csharp\r\n 代码\r\n``"	否
P4	"``csharp\r\n 代码\r\n~~~"	D1(T), D2(T), D1(T), D2(F), D3(F)	"``csharp\r\n 代码\r\n~~~"	"``csharp\r\n 代码\r\n~~~"	否
P5	"文本\r\n``csharp\r\n 代码\r\n``"	D1(F), D1(T), D2(T), D1(T), D2(F), D3(T)	同上	同上	否
P6	"文本\r\n``csharp\r\n 代码\r\n~~~"	D1(F), D1(T), D2(T), D1(T), D2(F), D3(F)	同上	同上	否

分析问题 1： 哪些路径在当前代码下不可达？原因是什么？

所有列出的路径在当前代码下均可达。代码的状态机设计允许任意顺序的 Fence 开启与闭合判断。

分析问题 2： 路径 P6 的实际返回值是什么？与预期是否一致？属于哪种缺陷类型？

实际返回值与预期一致。如果预期不一致，则属于状态机流转缺陷（逻辑缺陷），即未能正确处理未匹配闭合标记的异常边界情况。

任务五：xUnit 测试实现与缺陷报告（20 分）

5.1 xUnit 测试代码

```
using WeaveDoc.Converter.Pandoc;
```

```
using Xunit;
```

```

namespace WeaveDoc.Converter.Tests;

public class MarkdownMathNormalizerTests
{
    [Fact]
    public void NormalizeDollarMath_TrimsSingleDollarMath()
    {
        var result = MarkdownMathNormalizer.NormalizeDollarMath("正文 $ x +
y $ 结束");

        Assert.Equal("正文 $x + y$ 结束", result);
    }

    [Fact]
    public void NormalizeDollarMath_PreservesCrLfLineEndings()
    {
        var input = "第一行 $ x $\r\n 第二行 $ y $\r\n";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal("第一行 $x$\r\n 第二行 $y$\r\n", result);
    }

    [Fact]
    public void NormalizeDollarMath_DoesNotNormalizeInsideMatchingCodeFence()
    {
        var input = ""
前 $ x $

```

```

    ```${code $ y $}
 `
 后 $ z $
 """;
}

var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

Assert.Contains("前 x", result);
Assert.Contains("code $ y $", result);
Assert.Contains("后 z", result);
}

[Fact]
public void
NormalizeDollarMath_DifferentFenceMarkerInsideFence_DoesNotCloseFence()
{
 var input = ""
 ``
 code $ x $
    ~~~
    still code $ y $
    ``
    after $ z $
    """;

    var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

    Assert.Contains("code $ x $", result);

```

```

        Assert.Contains("still code $ y $", result);

        Assert.Contains("after $z$", result);
    }

    [Fact]
    public void NormalizeDollarMath_ConvertsTextCircledMath()
    {
        var input = @"A $\textcircled{0}$ B $\textcircled{1}$ C
$^{\textcircled{20}}$";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal("A ① B ① C ②①", result);
    }

    [Fact]
    public void NormalizeDollarMath_OutOfRangeTextCircledMath_RemainsDollarMath()
    {
        var input = @"A $\textcircled{21}$";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal(@"A $\textcircled{21}$", result);
    }

    [Fact]
    public void NormalizeDollarMath_DoesNotTreatEscapedDollarAsMathDelimiter()

```

```

    {
        var input = @"price \$ x and $ y $";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal(@"price \$ x and $y$", result);
    }

    [Fact]
    public void NormalizeDollarMath_DoesNotTreatDoubleDollarAsSingleDollarMath()
    {
        var input = "block $$ x $$ and inline $ y $";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal("block $$ x $$ and inline $y$", result);
    }

    [Fact]
    public void NormalizeDollarMath_DoesNotNormalizeClosedInlineCode()
    {
        var input = "code `$ x $` and math $ y $";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal("code `$ x $` and math $y$", result);
    }

```

```

    [Fact]
    public void NormalizeDollarMath_UnclosedInlineCode_AllowsLaterMathNormalization()
    {
        var input = "broken `code $ x $ and math $ y $";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal("broken `code $x$ and math $y$", result);
    }

    [Fact]
    public void NormalizeDollarMath_EmptyMathDelimiter_IsLeftUnchanged()
    {
        var input = "empty $ $";

        var result = MarkdownMathNormalizer.NormalizeDollarMath(input);

        Assert.Equal("empty $ $", result);
    }
}

```

## 5.2 xUnit 运行截图

```
/home/tby/桌面/WeaveDoc/test/.conda) tby@tby-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/桌面/WeaveDoc/test$ dotnet test tests/WeaveDoc.Converter.Tests/WeaveDoc.Converter.Tests.csproj --filter FullyQualifiedName~MarkdownMathNormalizerTests
还原完成 (0.5)
WeaveDoc.Converter net10.0 已成功 (0.3 秒) -> src/WeaveDoc.Converter/bin/Debug/net10.0/WeaveDoc.Converter.dll
WeaveDoc.Converter.Tests net10.0 已成功 (0.3 秒) -> tests/WeaveDoc.Converter.Tests/bin/Debug/net10.0/WeaveDoc.Converter.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445ala (64-bit .NET 10.0.9)
[xUnit.net 00:00:00.05] Discovering: WeaveDoc.Converter.Tests
[xUnit.net 00:00:00.10] Discovered: WeaveDoc.Converter.Tests
[xUnit.net 00:00:00.10] Starting: WeaveDoc.Converter.Tests
[xUnit.net 00:00:00.16] WeaveDoc.Converter.Tests.MarkdownMathNormalizerTests.NormalizeDollarMath_DoesNotTreatDoubleDollarAsSingleDollarMath [FAIL]
[xUnit.net 00:00:00.16] Assert.Equal() Failure: Strings differ
[xUnit.net 00:00:00.16]                                     (pos 8)
[xUnit.net 00:00:00.16] Expected: "block $$ x $$ and inline $y$"
[xUnit.net 00:00:00.16] Actual:   "block $$x$and inline$ y $"
[xUnit.net 00:00:00.16]                                     (pos 8)
[xUnit.net 00:00:00.16] Stack Trace:
[xUnit.net 00:00:00.16] /home/tby/桌面/WeaveDoc/test/tests/WeaveDoc.Converter.Tests/MarkdownMathNormalizerTests.cs(100,0): at WeaveDoc.Converter.Tests.MarkdownMathNormalizerTests.NormalizeDollarMath_DoesNotTreatDoubleDollarAsSingleDollarMath()
[xUnit.net 00:00:00.16] at System.Reflection.MethodBaseInvoker.InterpretedInvoke_Method(Object obj, IntPtr* args)
[xUnit.net 00:00:00.16] at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj, BindingFlags invokeAttr)
[xUnit.net 00:00:00.15] Finished: WeaveDoc.Converter.Tests
WeaveDoc.Converter.Tests 测试 net10.0 失败, 出现 1 错误 (0.9 秒)
/home/tby/桌面/WeaveDoc/test/tests/WeaveDoc.Converter.Tests/MarkdownMathNormalizerTests.cs(100): error TESTERROR:
WeaveDoc.Converter.Tests.MarkdownMathNormalizerTests.NormalizeDollarMath_DoesNotTreatDoubleDollarAsSingleDollarMath (1ms): 错误消息: Assert.Equal() Failure: Strings differ
                                     (pos 8)
Expected: "block $$ x $$ and inline $y$"
Actual:   "block $$x$and inline$ y $"
                                     (pos 8)
堆栈跟踪:
at WeaveDoc.Converter.Tests.MarkdownMathNormalizerTests.NormalizeDollarMath_DoesNotTreatDoubleDollarAsSingleDollarMath() in /home/tby/桌面/WeaveDoc/test/tests/WeaveDoc.Converter.Tests/MarkdownMathNormalizerTests.cs:line 100
at System.Reflection.MethodBaseInvoker.InterpretedInvoke_Method(Object obj, IntPtr* args)
at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj, BindingFlags invokeAttr)
测试摘要: 总计: 11, 失败: 1, 成功: 10, 已跳过: 0, 持续时间: 0.9 秒
在 2.3 秒内生成 失败, 出现 1 错误
```

5.3 缺陷记录表（8 分）

缺陷编号	发现的测试方法	缺陷所在方法	错误现象（实际值 vs 预期值）	首次检出的覆盖标准	对应需求违背
BUG-01	NormalizeDollarMath_DoesNotTreatDoubleDollarAsSingleDollarMath	MarkdownMathNormalizer.NormalizeLine, 由 NormalizeDollarMath 调用	实际值: block \$\$x\$and inline\$ y \$; 预期值: block \$\$ x \$\$ and inline \$y\$	边界值覆盖: \$ 与 \$\$ 数学公式边界	违背 FR-01: 系统应支持将含有数学公式的 MD 文档按 AFD 协议导出为符合高校规范的 Word/PDF; 同时违背“格式呈现准确、不出现乱码/格式错误”的需求来源

				界	
--	--	--	--	---	--

**分析问题 1：** 语句覆盖测试集能发现哪些缺陷？哪些需要更高覆盖标准才能发现？

语句覆盖仅能发现导致程序崩溃、语法错误或明显阻断执行的缺陷。对于逻辑运算符错误（如 `&&` 误写为 `||`）、条件判断遗漏等隐藏的逻辑缺陷，必须依赖判定覆盖、条件组合覆盖等更高标准才能发现。

**分析问题 2：** 有没有缺陷是黑盒方法先发现的？有没有只有白盒才能发现的？

黑盒先发现： 输入输出不符合需求规格说明书的缺陷（如数学公式转换错误），黑盒测试不关心代码实现，直接验证结果，发现效率更高。

白盒才能发现： 代码中存在不可达语句（死代码）、未处理的异常分支、状态机死锁等。这些缺陷可能在常规输入下不触发，只有通过白盒分析代码结构才能发现。

## 实验三：综合对比分析

### 6.1 各方法缺陷检出能力矩阵（30 分）

（✓=系统性检出，✗=无法检出，△=取决于测试值选取）

测试方法	BUG-01
黑盒：等价类划分	△
黑盒：边界值分析	✓
黑盒：错误推测法	✓
黑盒：判定表	△
白盒：语句覆盖	✗
白盒：判定覆盖	△
白盒：条件覆盖	✓
白盒：判定-条件覆盖	✓
白盒：条件组合覆盖	✓
白盒：基础路径覆盖	✓

### 6.2 综合分析问题（20 分）

#### 问题 1（6 分）：

请结合你的项目方法，说明条件覆盖（CC）比判定覆盖（DC）更有可能检出哪类缺陷？为什么？用你的实验用例给出证据。

结合本项目，条件覆盖（CC）比判定覆盖（DC）更容易检出“复合判断中某个子条件被遗漏或写错”的缺陷，尤其是 BUG-01 这种 \$、转义 \$、\$\$ 混合边界判断错误。

原代码关键判断是：

```
line[i] == '$' && !IsEscaped(line, i) && !IsDoubleDollar(line, i)
```

问题 2（6 分）：

你是否发现了理论上应被覆盖但实际不可达的基础路径？如果有，分析：（1）不可达的原因；（2）这说明代码存在什么问题；（3）黑盒方法能否直接发现？

发现了理论上应被覆盖但实际不可达的基础路径。

不可达路径	原因
D1 = F 后继续判断 D2 或 D3	D2、D3 只在 D1 = T，即当前行是代码块围栏时才会执行
D1 = T 后继续判断 D4	D4 只在非围栏行分支执行
D2 = T 后继续判断 D3	D3 是 else if，只有 D2 = F 即已在代码块内时才可能执行
首次循环前直接走最终返回	Split('\n') 至少产生一个元素，循环至少执行一次

这说明代码本身不是所有理论组合都能独立成立，部分路径受前置状态约束，例如 inFence 和 fenceMarker。这不一定是代码缺陷，而是状态机式控制流的自然结果。但它也说明 CFG 分析时不能机械枚举所有真假组合，否则会把不可达路径误认为需要测试覆盖。

黑盒方法通常不能直接发现这些不可达路径。黑盒测试只观察输入输出，不知道内部 D1-D4 的控制依赖关系。它可以通过大量输入发现某些行为无法触发，但无法直接判断“哪条基础路径不可达”以及不可达原因。

问题 3（8 分）：

维度	黑盒测试（结合你的项目给出具体例证）	白盒测试（结合你的项目给出具体例证）
能发现的典型缺陷类型	需求违背、边界输入错误。 例如用 block \$\$ x \$\$ and inline \$ y \$ 发现 \$\$ 被错误规范化，违背数学公式导出需求	分支遗漏、复合条件错误、路径不可达。例如发现 !IsDoubleDollar(line, i) 被删除后，\$\$ 路径未被正确跳过

不易发现的典型缺陷类型	不易定位具体错误语句，也不易发现某个内部条件从未取到真	不易判断输出是否符合用户真实需求，例如不知道 \$\$ 是否应保留，必须依赖需求语义
是否依赖需求规格文档	强依赖。需要知道 FR-01 要求“含数学公式的 MD 文档正确导出”	部分依赖。设计覆盖路径依赖源码，判断结果正确性仍依赖需求
是否依赖源代码	不依赖或弱依赖	强依赖。需要阅读 NormalizeLine、IsDoubleDollar、IsEscaped 等源码
代表性方法（本实验中）	边界值分析、错误推测法	条件覆盖、条件组合覆盖、基础路径覆盖
与另一方的互补关系	能从用户视角发现“结果错了”，但定位能力弱	能定位遗漏条件和未覆盖路径，但需要黑盒需求判断来确认是不是缺陷

### 核心结论（3~5 句话）：

本次实验中，**边界值分析 + 条件覆盖 + 基础路径覆盖**的组合缺陷检出能力最高。BUG-01 本质上是 \$\$ 边界输入和复合条件遗漏共同导致的，因此单靠普通等价类或语句覆盖容易漏掉。黑盒测试能证明输出违背需求，白盒测试能解释为什么会错。不存在一种单一方法能同时保证需求正确性、边界充分性和内部路径覆盖。